
OPTIMISING MOE AND SPARSE ATTENTION LAYERS ON BLACKWELL GPU

Arun Reddy Anugu^{*} Rohith Rongali^{*} Kartik Agarwal^{*}

ABSTRACT

In our project we propose to iteratively accelerate MoE and Sparse Attention layers in DeepSeek V3.2 on Blackwell GPUs from the Flashinfer challenge. We aim to do this via kernel fusion, batching and other blackwell specific optimisations.

1 INTRODUCTION AND PROBLEM STATEMENT

The rapid evolution of Large Language Models (LLMs) has popularized complex architectures like DeepSeek V3.2, which utilizes Mixture-of-Experts (MoE) and DeepSeek Sparse Attention (DSA) for high efficiency. However, deploying these models on NVIDIA Blackwell B200 hardware introduces severe system bottlenecks. The primary challenge has shifted from raw computation to massive data movement, where existing kernels suffer from high register pressure and pipeline stalls during token routing and sparse Key-Value (KV) cache management. To resolve these inefficiencies, we are developing custom Blackwell-native kernels for both MoE and DSA.

2 STATE-OF-THE-ART METHODS

The current state-of-the-art for LLM inference on NVIDIA hardware is defined by libraries such as FlashInfer, DeepGemm, and NVIDIA’s TransformerEngine. FlashInfer provides the competition’s baseline, offering highly optimized CuTe-based kernels for MoE and attention. DeepGemm, developed by DeepSeek-AI, introduced JIT-compiled kernels that handle the irregular Grouped-GEMM shapes characteristic of V3.2. For DeepSeek Sparse Attention, the official FlashMLA github repository is provided.

3 FUTURE DIRECTIONS

3.1 Mixture of Experts (MoE)

The baseline/naive implementation suffers from several key inefficiencies: eager dequantization of all expert weights upfront regardless of token occupancy; sequential per-expert GEMM launches with no SM packing; redundant token

gather/scatter operations, one per expert, as separate global memory passes; a standalone SwiGLU activation that forces the GEMM1 output through DRAM unnecessarily; and repeated materialization of intermediate logit tensors during routing. We aim to address these by (1) fusing the entire routing pipeline into a single kernel that never writes intermediate logit tensors to DRAM, (2) performing a single upfront token permutation by expert assignment and issuing a single Grouped-GEMM over all active experts, reducing gather/scatter from one pass per expert to a single pair and (3) If time permits we also plan to fuse GEMM1 $\rightarrow$ SwiGLU $\rightarrow$ GEMM2 within a single persistent kernel so that the intermediate activation tensor is kept in shared memory rather than written to DRAM between operations. The dequantization overhead is largely subsumed by the Blackwell-specific optimizations described below, where NVFP4 dequantization is hardware-fused directly into the MMA instruction.

Blackwell-specific optimizations: We aim to exploit two hardware primitives introduced in SM 10.0. First, the Tensor Memory (TMEM) unit provides a hardware-managed asynchronous copy engine that is fully decoupled from warp execution, our kernel issues prefetches for upcoming weight and activation tiles speculatively, and the hardware delivers them in the background while Tensor Cores compute the current tile. This achieves a 3–4 stage pipeline with no manual software orchestration, unlike Hopper where analogous overlap required careful cp.async choreography. Second, NVFP4 weight quantization halves weight memory bandwidth relative to FP8, with dequantization fused directly into the MMA instruction in hardware at zero software cost. If time permits we also plan to work on utilising the Thread block clusters for weight broadcasting via distributed shared memory.

Batch size threshold: The utility of Grouped-GEMM batching differs fundamentally between prefill and decode. During prefill, most experts receive a meaningful sub-batch of tokens, making batching effective at amortizing launch over-

^{*}Equal contribution . Correspondence to: ... <...>.

head, packing SM work across experts, and making GEMM fusion impactful. However during decode, most experts receive zero or one token per step, making the GEMMs too small for batching to help — here the primary levers are the Blackwell-specific optimizations rather than batching itself. We also plan to highlight this tradeoff in our experiments.

3.2 DeepSeek Sparse Attention (DSA)

DeepSeek V3.2 uses Multi-head Latent Attention (MLA), a low-rank KV compression scheme where the attention score is computed as the sum of two dot products: a primary term over a compressed 512-dimensional KV representation and a 64-dimensional positional correction term. Rather than attending over the full context, each query pre-selects the top-2048 most relevant KV tokens via a learned routing mechanism, stored in a paged structure with page size 64. The reference implementation processes tokens serially, issues two separate GEMMs per token for the split dot products, explicitly gathers KV tokens as a standalone DRAM pass, and fully materializes the attention weight matrix between softmax and output projection — all of which we aim to address.

We propose to optimize this via progressive steps: First, we will fuse the two dot products into a single GEMM by concatenating $[q_{nope}|q_{pe}]$ and $[Kc|Kp]$ along the head dimension, yielding an equivalent 576-dimensional dot product at no accuracy cost. Second, we will replace the serial per-token loop with a batched GEMM over all tokens simultaneously, analogous to our Grouped-GEMM approach in MoE, eliminating launch overhead and enabling SM packing. Third, we plan to implement a FlashAttention-style online softmax that tiles over the 2048 selected KV tokens in chunks, carrying running accumulators in registers and never materializing the full attention weight matrix to DRAM. Fourth, we will sort sparse indices by page prior to execution to coalesce KV memory accesses at page granularity. If time permits, we additionally plan to fuse the KV gather into the compute pipeline using TMA, replacing the explicit gather pass with hardware-managed prefetches driven directly by the sparse index list.

4 EVALUATION OF PERFORMANCE

We evaluate our Blackwell-native MoE kernel against state-of-the-art grouped-GEMM implementations, such as the FlashInfer baseline. MoE is primarily evaluated under Pre-fill workload and DSA is evaluated under Decode workload.

4.1 Metrics

We measure the following system-level metrics: End-to-end MoE layer latency (ms), Throughput (tokens/sec), Kernel launch count, Achieved DRAM bandwidth (GB/s), SM

occupancy and register utilization, Numerical correctness (max/mean absolute error vs reference)

5 WORKLOAD DISTRIBUTION AMONG TEAM MEMBERS

Each team member will lead one stage while collaborating during integration and evaluation. All members will collaborate on benchmarking, debugging, and final performance analysis.

Member 1: Naive MoE Kernel Implementation Implement a baseline grouped-GEMM MoE layer with separate routing, packing and Sparse Attention kernels.

Member 2: State-of-the-Art Style Optimization Implement optimized token packing and grouped-GEMM scheduling inspired by existing libraries (e.g., FlashInfer-style execution).

Member 3: Blackwell-Specific Optimization Adapt the optimized kernel to leverage Blackwell architectural features.

6 TIMELINE AND MILESTONES

To ensure the successful delivery of the optimized kernels, we have divided our roadmap into four two-week sprints:

Milestone 1 (Weeks 1-2): Baseline & Profiling. Establish the reference implementation for DeepSeek V3.2 MoE and DSA layers. Conduct initial profiling on Blackwell GPUs to identify the exact cycle counts for stall reasons.

Milestone 2 (Weeks 3-4): Fusion & Grouped-GEMM. Implement the single-pass token permutation and fused routing kernels. Integrate Grouped-GEMM to replace sequential expert launches. For DSA, implement the concatenated dot-product fusion.

Milestone 3 (Weeks 5-6): Blackwell Native Features. Integrate SM 10.0 primitives. This includes replacing manual `cp.async` with TMEM-based asynchronous copies and implementing hardware-fused NVFP4 dequantization.

Milestone 4 (Weeks 7-8): Final Optimization & Benchmarking. Perform iterative refinement of tiling strategies and register pressure management. Conduct a comparative analysis against FlashInfer and FlashMLA.

7 CPU, GPU AND STORAGE INFRASTRUCTURE

We plan to use the Blackwell GPUs provided by Modal.